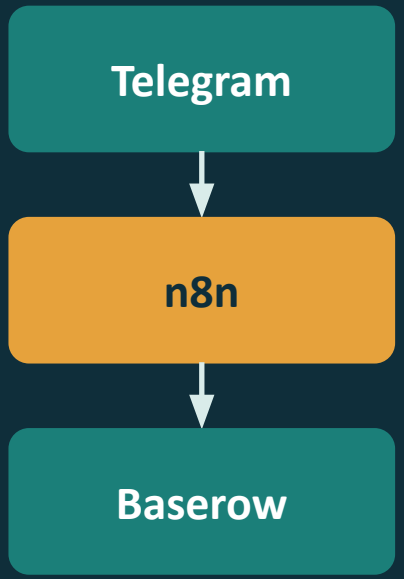
Seguimiento y clasificación automática de leads



Inmonova S.A.S.

Automatización inteligente de procesos con n8n, EAM 2026 II

Entrega del segundo corte

Javier Leonardo Argoty Roa | Simón David Cruz Suazo
Johan Steven Hernández Torres | Luis Eduardo Sinisterra Montaño

Proyecto e integrantes

Qué construyeron

Un flujo en n8n que recibe leads inmobiliarios por Telegram, los valida, los clasifica por nivel de interés y los guarda en una tabla de Baserow.

A quién beneficia

Al asesor comercial de Inmonova, que ve en un solo lugar qué lead atender primero.

Integrante	Rol principal
Javier Leonardo Argoty Roa	Desarrollo del workflow
Luis Eduardo Sinisterra Montaño	Pruebas
Simón David Cruz Suazo	Documentación
Johan Steven Hernández Torres	Bóveda de archivos y bot de Telegram

Los roles rotaron entre clases; se indica el rol con el que cada integrante figura en la ficha inicial y en la última bitácora.

Los leads se enfrían antes de recibir respuesta

1 Canales dispersos

Los leads llegan por formulario web, WhatsApp y teléfono, sin un proceso central.

2 Sin prioridad clara

El asesor decide a criterio propio a quién responder primero.

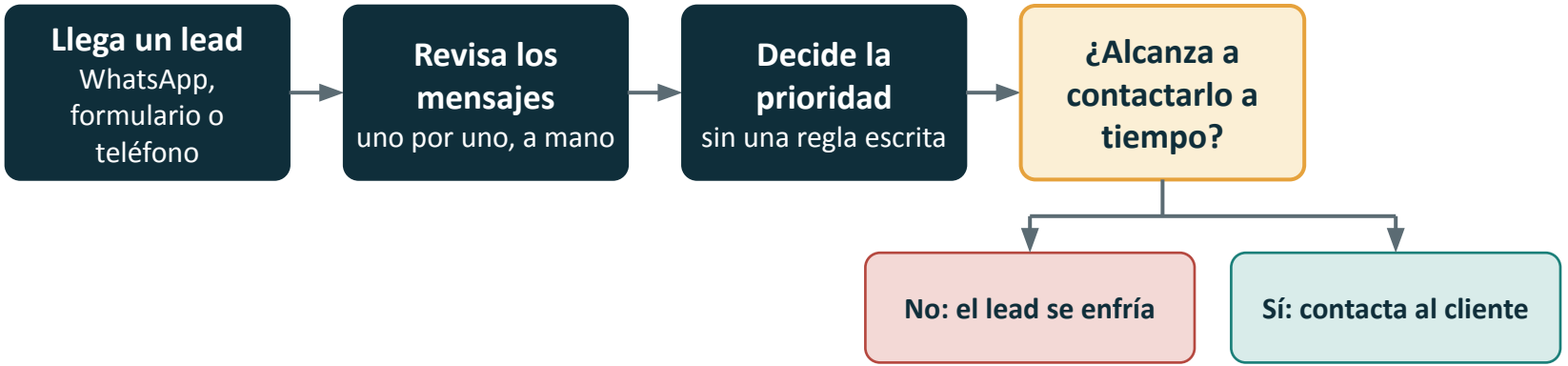
3 Respuesta tardía

Un lead de interés alto que espera 24 horas pierde probabilidad de conversión.

Situación: Camila Restrepo, de una constructora, escribe que quiere agendar una visita al apartamento hoy mismo. Su mensaje queda entre otros chats y nadie sabe que es el más urgente.

La pérdida de probabilidad por esperar 24 horas es la hipótesis de la ficha del proyecto; el equipo no la midió.

Proceso manual: el asesor prioriza a criterio propio



Demoras
La respuesta depende de cuándo el asesor revisa los chats.

Errores
Sin registro único: leads sin clasificar o sin datos completos.

Tareas repetitivas
Leer, ordenar y anotar cada lead a mano.

Objetivo y alcance de esta versión

Clasificar cada lead al llegar y dejarlo registrado con la acción de seguimiento que le corresponde, sin intervención humana.

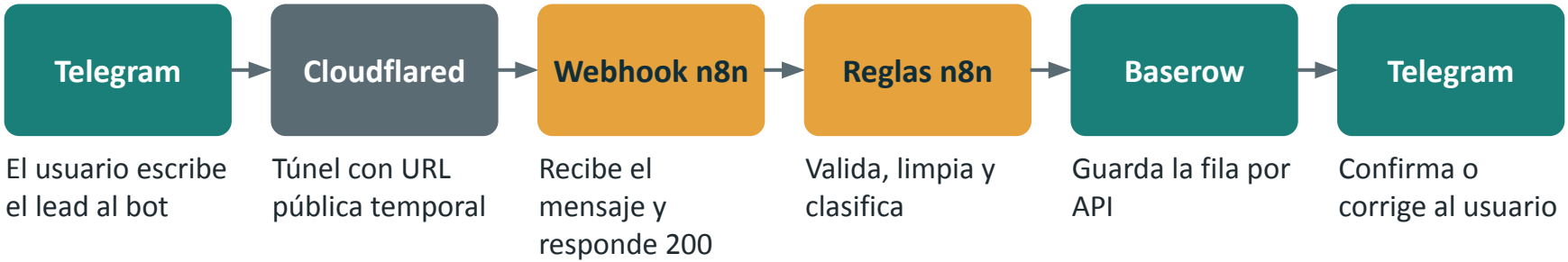
Incluye

- Entrada por webhook de Telegram
- Validación del formato del mensaje
- Clasificación por interés y tipo de correo
- Registro del lead en Baserow por API
- Confirmación al chat con la acción

No incluye todavía

- Aviso al asesor por correo o Slack
- Entrada por WhatsApp Business
- Evitar duplicados de un mismo mensaje
- Manejo de errores técnicos con reintentos
- IA para clasificar el interés

Telegram envía, n8n clasifica, Baserow guarda



Términos de esta solución

Webhook: dirección de n8n que Telegram llama cada vez que llega un mensaje.

API: canal por el que un programa lee o escribe datos en otro; aquí, n8n crea filas en Baserow.

Túnel: conexión que hace visible desde internet el n8n que corre en un computador local.

Un caso completo: Camila Restrepo, interés ALTO

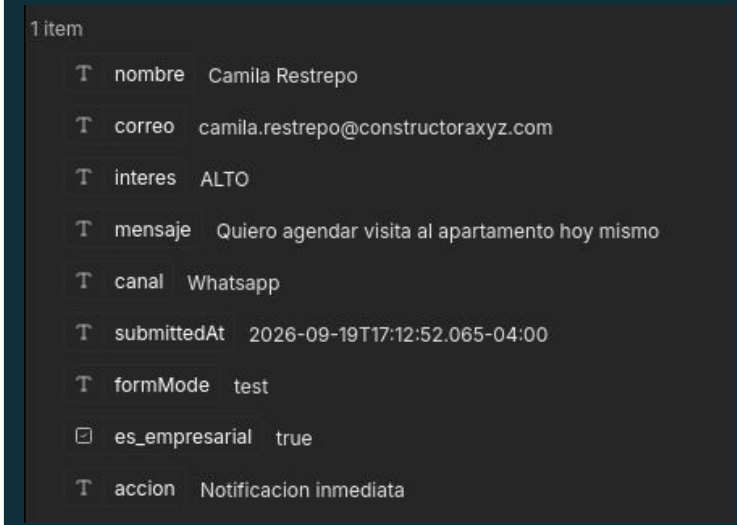
1. **Entrada:** Camila Restrepo | camila.restrepo@constructoraxyz.com | ALTO | Quiero agendar visita al apartamento hoy mismo

2. **Limpieza:** Separa por | , pone el correo en minúsculas y el interés en mayúsculas

3. **Validación:** Cumple las 5 condiciones: nombre, correo con @, interés válido, mensaje y chat

4. **Decisiones:** Dominio no es gmail, hotmail ni outlook: es_empresarial = true. Interés ALTO: acción inmediata

5. **Resultado:** Fila en Baserow y respuesta al chat con el nombre y la acción



Salida real del flujo: es_empresarial true y accion Notificacion inmediata.

Captura del 19 sep 2026 en la versión con formulario, que usa la misma clasificación del workflow final. El canal de entrada difiere.

Tres servicios externos y una llamada a API

Servicio	Para qué se usa	Cómo se conecta
Telegram Bot API	Recibe el lead escrito por el usuario y devuelve el aviso	Webhook de entrada y nodos Telegram, con credencial propia
Cloudflare Tunnel	Hace accesible el n8n local desde internet	Script que abre el túnel y registra el webhook con setWebhook
Baserow	Guarda cada lead clasificado	Nodo HTTP Request con credencial de tipo token

HTTP Request a Baserow

```
POST https://api.baserow.io/api/database/rows/table/{id_tabla}/?user_field_names=true
```

Envía 9 campos: nombre, correo, interes, mensaje, canal, submittedAt, formMode, es_empresarial y accion. El token vive en la credencial de n8n, no en el JSON.

Un workflow activo y cuatro versiones anteriores

Archivo	Qué hace	Nodos	Estado
Workflow_01_DatosValidacion	Datos simulados y dos If de validación	7	Historial
Workflow_02_LogicaControlFlujo	10 leads simulados, Filter, If, Merge y Switch	13	Historial
Workflow_03_Formulario	Entrada por formulario y limpieza de datos	10	Historial
Workflow_04_API	Agrega el POST a Baserow	11	Historial
Workflow_05_WebhookTelegram	Entrada por Telegram, validación, clasificación, registro y respuesta	15	**Entregado**

La solución no usa subworkflow ni manejador de errores aparte: toda la lógica está en el workflow 05. Separar la clasificación en un subworkflow queda propuesto.

Tres expresiones que hacen el trabajo

Separar el mensaje

```
texto.split('|').map(p => p.trim())
```

Entrada: Laura | LAURA@Mail.com | alto | Quiero visita

Salida: correo laura@mail.com, interes ALTO

Validar el interés

```
['ALTO','MEDIO','BAJO'].includes($json.interes)
```

Entrada: ALTO

Salida: true. Con ALT da false y el bot pide el formato

Armar el cuerpo del POST

```
JSON.stringify({ nombre: $json.nombre, ... })
```

Entrada: los campos ya clasificados

Salida: JSON válido, con es_empresarial como booleano

Validaciones y errores: lo que cubre el flujo y lo que no

Situación	Tipo	Qué ocurre	Estado
Mensaje sin los 4 campos, correo sin @ o interés fuera de lista	Regla del negocio	No se guarda. El bot responde con el formato esperado	**Implementado**
Telegram reintenta el mismo mensaje	Técnico	n8n responde 200 antes de procesar, así no hay reintento	**Implementado**
El mismo mensaje se envía dos veces	Negocio	Genera dos filas: no se deduplica por update_id	**Pendiente**
Baserow o Telegram fallan, o el token es inválido	Técnico	El nodo falla y la ejecución queda en el historial de n8n. No hay reintentos, manejador ni aviso	**Pendiente**

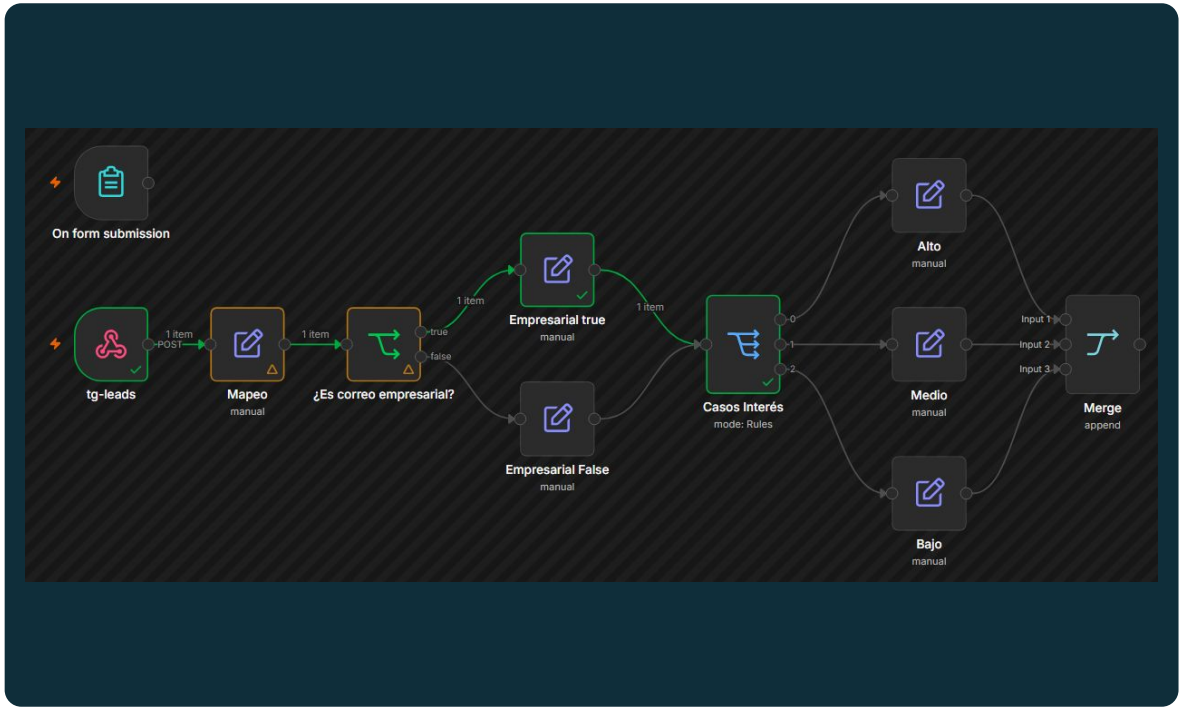
Un lead rechazado por formato no es un error técnico: el flujo termina bien y avisa al usuario. Un fallo de API sí lo es y hoy no se recupera solo.

Pruebas realizadas y resultados observados

Caso	Entrada ficticia	Resultado observado	Evidencia
Normal, ALTO	Camila Restrepo, correo de empresa	**es_empresarial true, Notificacion inmediata**	Captura de salida
Normal, MEDIO	Andrés Gómez, correo gmail	**es_empresarial false, seguimiento en 2 o 3 días**	Captura de salida
Correo no válido	Lead con correo sin @ (versión 01)	**Sale por la ruta Correo no valido**	Captura del canvas
Formato incorrecto por Telegram	Texto con menos de 4 campos	**Sin evidencia de ejecución**	Pendiente
Fallo técnico controlado	Token o API inválidos	**No se probó**	Pendiente

Pruebas manuales en n8n local. Las capturas de los casos normales vienen de la versión con formulario. No hay captura de la fila en Baserow.

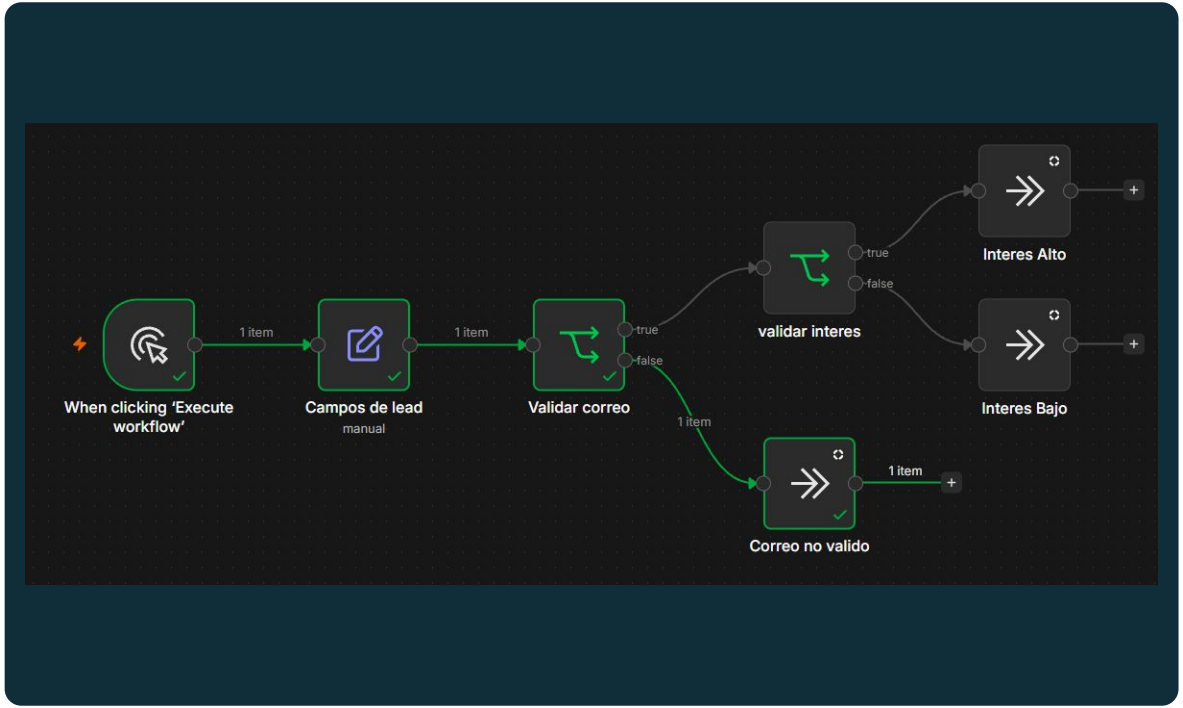
Evidencia 1: el webhook recibe el lead



El nodo tg-leads recibe un POST y entrega 1 item al flujo, que sigue hacia la clasificación.

Captura de una versión intermedia: el Form Trigger aún aparece en el lienzo.

Evidencia 2: un correo inválido se descarta



Un lead con correo sin @ toma la salida false y termina en el nodo Correo no valido.

Captura de la versión 01 del workflow, con datos simulados.

Evolución del proyecto desde el primer corte

Clase 02
Manual Trigger, Edit Fields e If

Clase 03
Filter, Switch y Merge con 10 leads

Clase 04
Form Trigger y limpieza de datos

Clase 05
HTTP Request a Baserow

Clase 06
Webhook de Telegram

Dificultades resueltas

- Un Merge faltante daba dos resultados separados
- Telegram repetía el mensaje y duplicaba leads: se responde 200 primero

Uso de IA

Claude, ChatGPT y OpenCode ayudaron a diagnosticar y a documentar. Cada propuesta se probó en n8n y se ajustó a los datos del equipo.

Estado final: implementado, simulado y propuesto

Implementado

- Webhook POST tg-leads con respuesta 200 inmediata
- Parseo y validación de 5 condiciones
- Clasificación por correo e interés
- Registro en Baserow por HTTP Request
- Confirmación al chat con la acción

Simulado

- Notificacion inmediata es un valor del campo accion, no un aviso al asesor
- Los seguimientos de 24 a 72 horas son solo etiquetas
- formMode se envía fijo como test

Propuesto

- Manejador de errores y reintentos
- Subworkflow de clasificación
- Deduplicar por update_id
- Aviso al asesor por correo o Slack
- IA para clasificar el interés

Beneficios esperados, no medidos: menor tiempo de respuesta y registro único. El equipo no midió ahorros ni conversión.

Cómo poner la solución en marcha

1. Importar Workflow_05_WebhookTelegram.json en n8n
2. Crear las credenciales telegramApi y baserowTokenApi con cuenta propia y reasignarlas en los nodos
3. Crear en Baserow la tabla con las 9 columnas y poner su id en la URL del nodo HTTP
4. Crear el bot con BotFather y guardar TG_TOKEN en un archivo .env (ver .env.example)
5. Activar el workflow y ejecutar el script que abre el túnel y registra el webhook
6. Escribir al bot: Nombre | correo | ALTO | mensaje, y revisar la fila y la respuesta
7. Al terminar: Ctrl+C cierra el túnel y se desactiva el workflow

La URL del túnel es temporal: cambia en cada arranque y puede dejar de funcionar. El script la vuelve a registrar.

Acceso al proyecto

Repositorio en GitHub: *https://github.com/Argoty/SeguimientoLeads-N8N*

```
Nombre_Proyecto/
  README.md
  workflows/
    Workflow_05_WebhookTelegram.json
  documentacion/
    Bitacora_Unificada_Proyecto.md
    Presentacion_Corte_2.pdf
    diagramas/
    evidencias/
```

Desde el README se abren

- La bitácora unificada
- Esta presentación en PDF
- Los JSON de los workflows
- Los diagramas y las evidencias

Si es privado, dar acceso al docente.